Julius-Maximilians-
UNIVERSITÄT
WÜRZBURG

Lehrstuhl für Informatik VIII
Informationstechnik für Luft- und Raumfahrt
Prof. Dr.-Ing. Sergio Montenegro

Introduction to RODOS

Online-Floatsat

Summer Term 2021

Frank Flederer

E-Mail: frank.flederer@uni-wuerzburg.de

Tel.: +49 931 31-85120

Inter Thread Communication

- Purposes
  - Exchange Data
  - Synchronize Threads
- Different approaches

Suspend / Resume

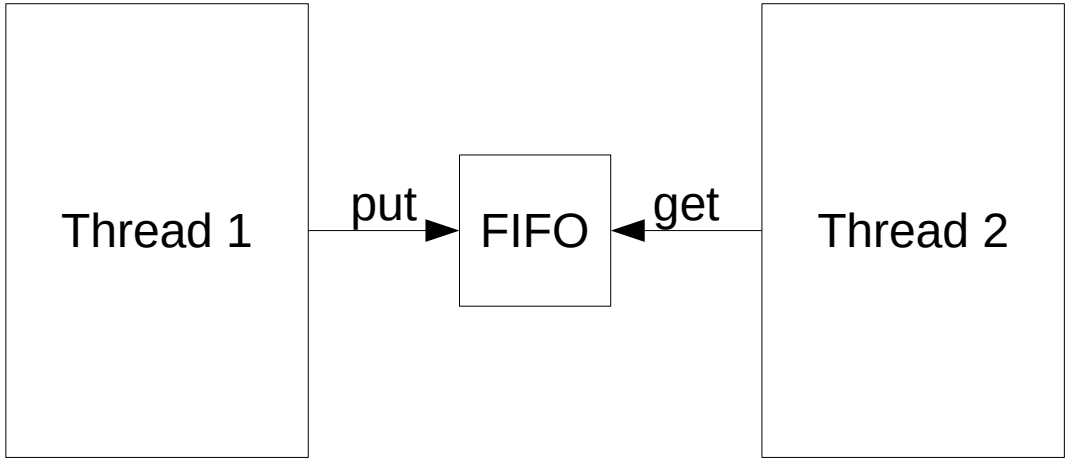
```
class ConsumerThread : public RODOS::StaticThread<> {
public:
  void run() override {
    while(true) {
      RODOS::AT(RODOS::END_OF_TIME);
      RODOS::PRINTF("Consuming\n");
    }
  }
};

ConsumerThread consumerThread

class ProducerThread : public RODOS::StaticThread<> {
public:
  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      RODOS::AT(RODOS::END_OF_TIME);
      RODOS::PRINTF("Producing\n");
      consumerThread.resume();
    }
  }
};

ProducerThread producerThread
```

- Useful for interrupt handling
- Disadvantage: tight coupling

Suspend / Resume for interrupts

```
class ConsumerThread : publicRODOS::StaticThread<> {
  uint32_t dataToConsume;

public:
  void setData(uint32_t newData) {
    dataToConsume = newData;
  }

  void run() override {
    while(true) {
      RODOS::AT(RODOS::END_OF_TIME);
      RODOS::PRINTF("Consuming \n", dataToConsume);
    }
  }
};

ConsumerThread consumerThread

void someInterruptServiceRoutine() {
  consumerThread.setData(readData());
  consumerThread.resume();
}
```

FIFO



FIFO

- First in first out
  - `bool put(const Type& val)`
    - returns true if the value is stored into the FIFO, false if the FIFO is full
  - `bool get(Type& val)`
    - returns true if a value is read from the FIFO, false if the FIFO is empty
    - passed parameter is overwritten
  - … and other useful methods. Have a look at the header file.

FIFO

```
RODOS::Fifo<uint32_t, 16> fifo;

class ConsumerThread : public RODOS::StaticThread<> {
public:
  void run() override {
    uint32_t data;
    TIME_LOOP(200*MILLISECONDS, 500*MILLISECONDS) {
      if(fifo.get(data)) {
        // process data
      }
    }
  }
};

ConsumerThread consumerThread

class ProducerThread : public RODOS::StaticThread<> {
public:
  uint32_t data;

  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      fifo.put(data);
    }
  }
};

ProducerThread producerThread
```

- Process data with a buffer
- Without implicit suspension (return immediately)

Synchronized FIFO

- Extended FIFO with blocking methods
  - `bool syncPut(const Type &val, const int64_t timeout = END_OF_TIME)`
    - if the FIFO is full, the function returns at the latest after the given timeout
  - `bool syncGet(Type &val, const int64_t timeout = END_OF_TIME)`
    - if the FIFO is empty, the function returns at the latest after the given timeout

Synchronized FIFO

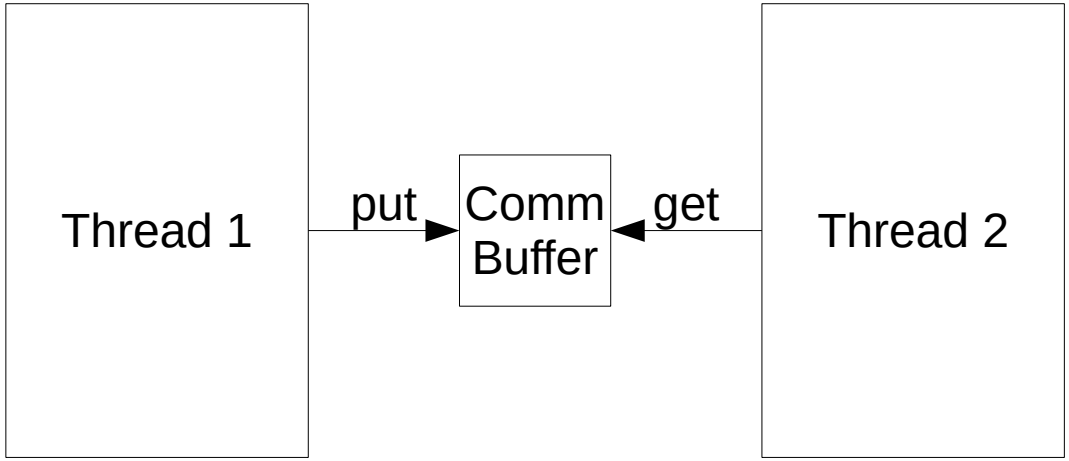
```
RODOS::SyncFifo<uint32_t, 16> fifo;

class ConsumerThread : public RODOS::StaticThread<> {
public:
  void run() override {
    uint32_t data;
    while(true) {
      fifo.syncGet(data, RODOS::END_OF_TIME);
      // process data
    }
  }
};

ConsumerThread consumerThread

class ProducerThread : public RODOS::StaticThread<> {
public:
  uint32_t data;

  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      fifo.syncPut(data);
    }
  }
};

ProducerThread producerThread
```

- Automatic suspension
- Useful when a thread‘s purpose is processing data as it comes into the system
- Suitable for Interrupt handling

CommBuffer

- Contains one value
- Accesses are thread-safe
  - `void put(const Type& data)`
  - `void get(Type& data)`
    - passed parameter is overwritten
  - `bool getOnlyIfNewData(Type& data)`
    - returns true if new data is available, falls if data is old
    - the passed paramer is only overwritten if the data is new

CommBuffer



CommBuffer

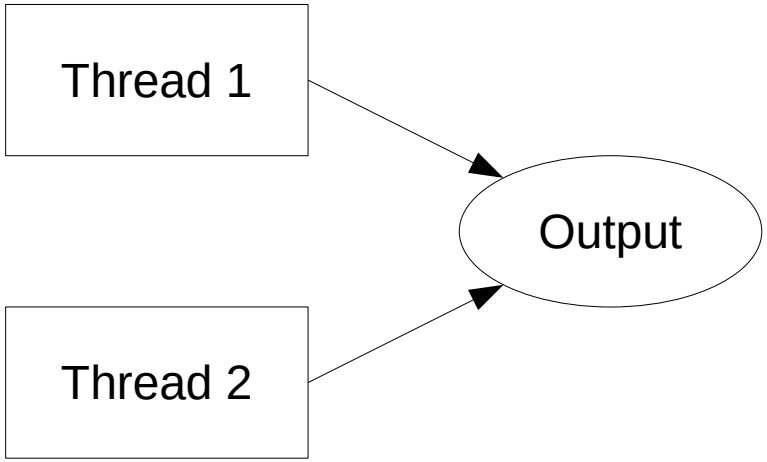
```
RODOS::CommBuffer<uint32_t> buffer;

class ConsumerThread : public RODOS::StaticThread<> {
public:
  void run() override {
    uint32_t data;
    TIME_LOOP(200*MILLISECONDS, 500*MILLISECONDS) {
      buffer.get(data);
      // process data
    }
  }
};

ConsumerThread consumerThread

class ProducerThread : public RODOS::StaticThread<> {
public:
  uint32_t data;

  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      buffer.put(data);
    }
  }
};

ProducerThread producerThread
```

- Process only the latest data
- Useful if only the latest value is important, not the history

Semaphore

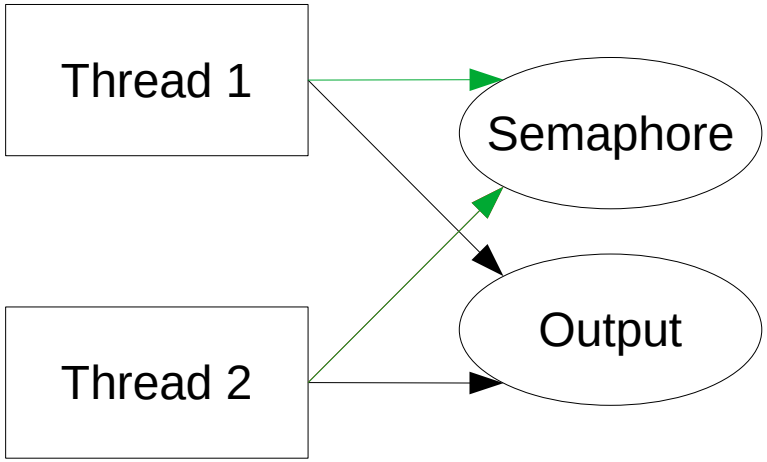
- Suspends threads for different purposes
  - Mutexes
  - Signaling
- A semaphore can be entered only once. All other threads are suspended until the entered thread leaves the semaphore

Semaphore



- thread 1 intends to write "ABC"
- thread 2 intends to write "123"
- output could be "A12B3C"
  - receiver does not understand anything
  - we have to make sure only one thread is printing at a given time

Semaphore



- the threads have to lock/enter the semaphore before writing data
- the semaphore ensures that only one thread can lock/enter at a given time

Semaphore

```
RODOS::Semaphore semaphore;

class MyFirstThread : public RODOS::StaticThread<> {
public:
  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      semaphore.enter();
      // access the device
      semaphore.leave();
    }
  }
};

MyFirstThread myFirstThread

class MySecondThread : public RODOS::StaticThread<> {
public:
  void run() override {
    TIME_LOOP(1*SECONDS, 1*SECONDS) {
      PROTECT_IN_SCOPE(semaphore);
      // access the device
    }
  }
};

MySecondThread mySecondThread
```

- For example: accessing a device by multiple threads

Enter and leave the semaphore manually.

The semaphore is entered immediately and left automatically as soon as the scope is left

Error Handling

- Rodos does not use exceptions (just as many embedded systems)
- Instead: errors are reported via return values
- Combination of value OR error: RODOS::Result

Error Code

- Represented as uint32_t
  0 – 999: predefined error values
  1000 – 1.999: reserved
  10000 and up: user defined error values
- ErrorCode::OK indicates that no error ocurred

Result

- Contains two variables: val(ue) and err(or)

```
ErrorCode err;
Type      val;
```

- Implicit constructors for ErrorCode and Value

```
Result(const Type& v)      : err(ErrorCode::OK), val(v) {}
Result(const ErrorCode& e) : err(e) {}
```

- Check for error

```
bool    isOk() const                { return err == ErrorCode::OK; }
```

- Get and set error codes as integer value

```
int32_t getErr() const              { return (int32_t)err; }
void    setErr(int32_t iErr)        { err = (ErrorCode)iErr; }
```

Error Handling / Result

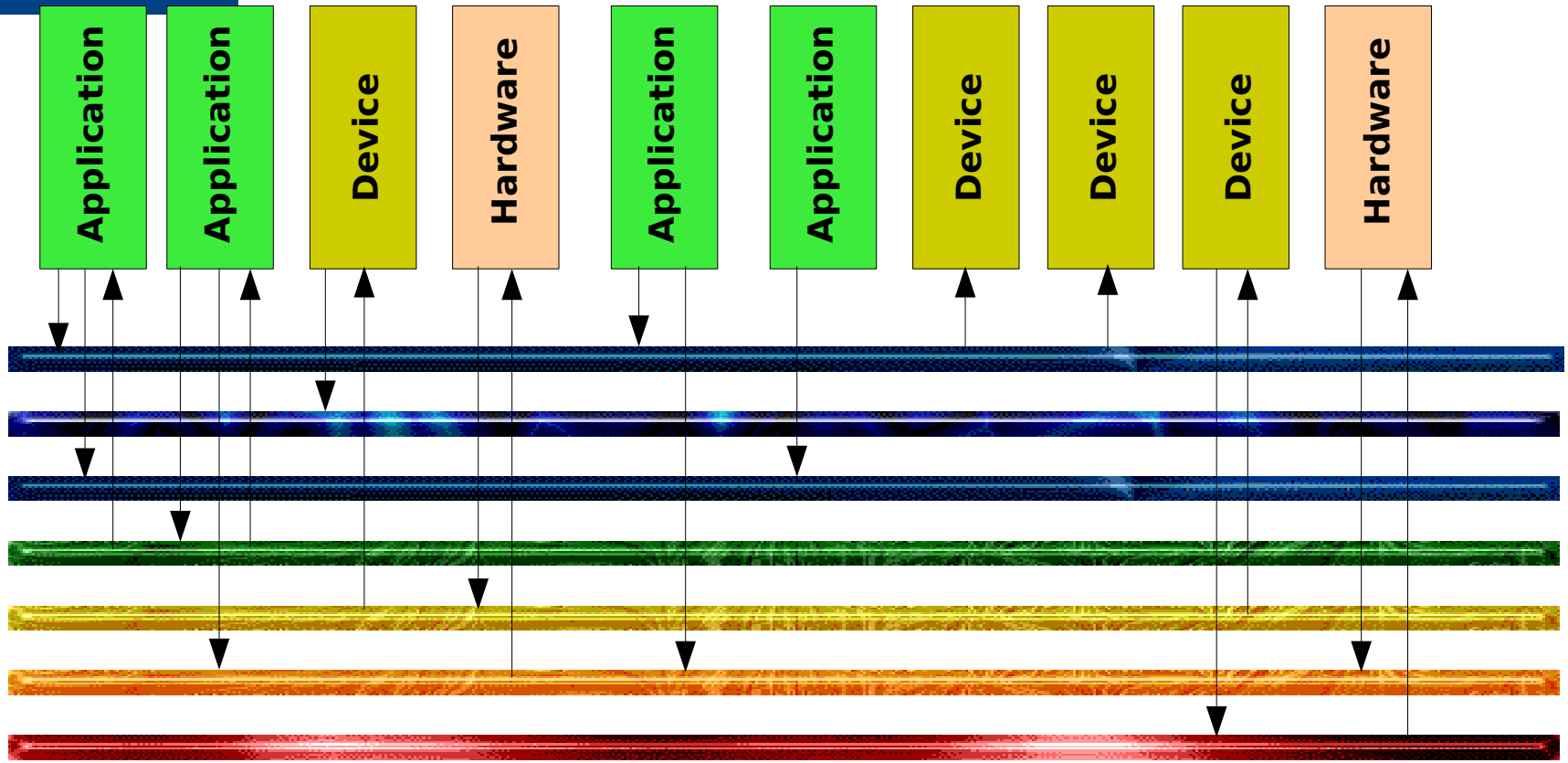
```
RODOS::Result<int> myCalledFunction() {
  if(isMyDeviceReady()) {
    return readFromMyDevice();
  } else {
    return RODOS::ErrorCode::NO_RESULT;
  }
}

void myCallingFunction() {
  RODOS::Result result = myCalledFunction();
  if(result.isOk()) {
    RODOS::PRINTF("Result: %d\n",
      result.val);
  } else {
    RODOS::PRINTF("Error: %d\n",
      result.getErr());
  }
}
```

- Called function can just return an error code or a value
- Calling function has to check if the result is OK or not

Publish / Subscribe

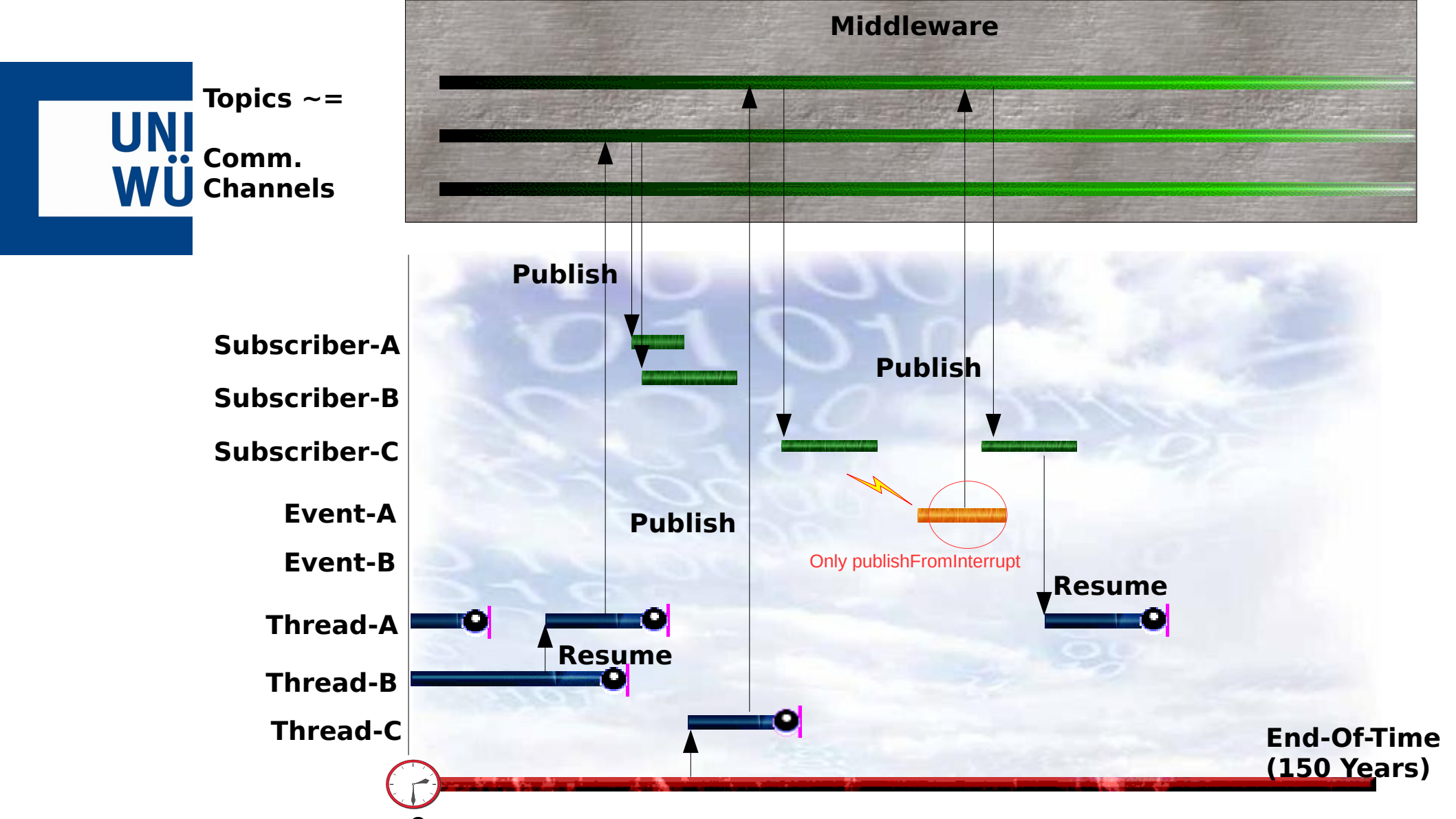
- Topics represent communication channels with an ID and a given data type
- Subscribers register for topics
- Publishers send messages to topics
- Published messages are forwarded to all subscribers

Publish / Subscribe

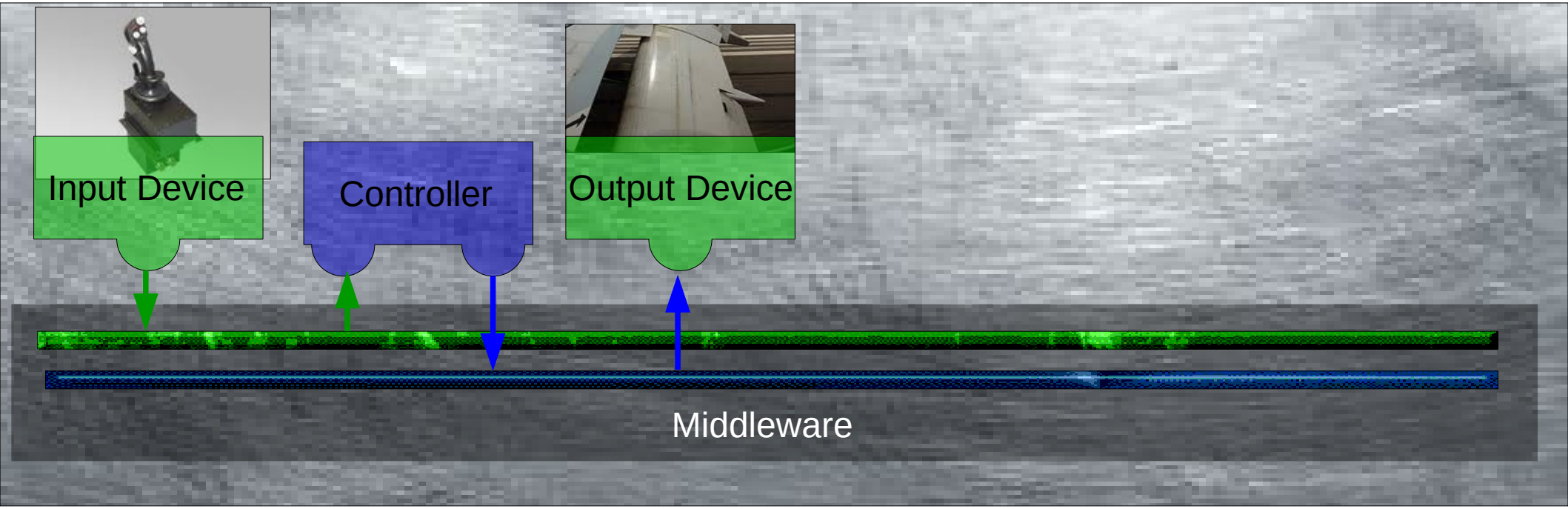


Topics ~=

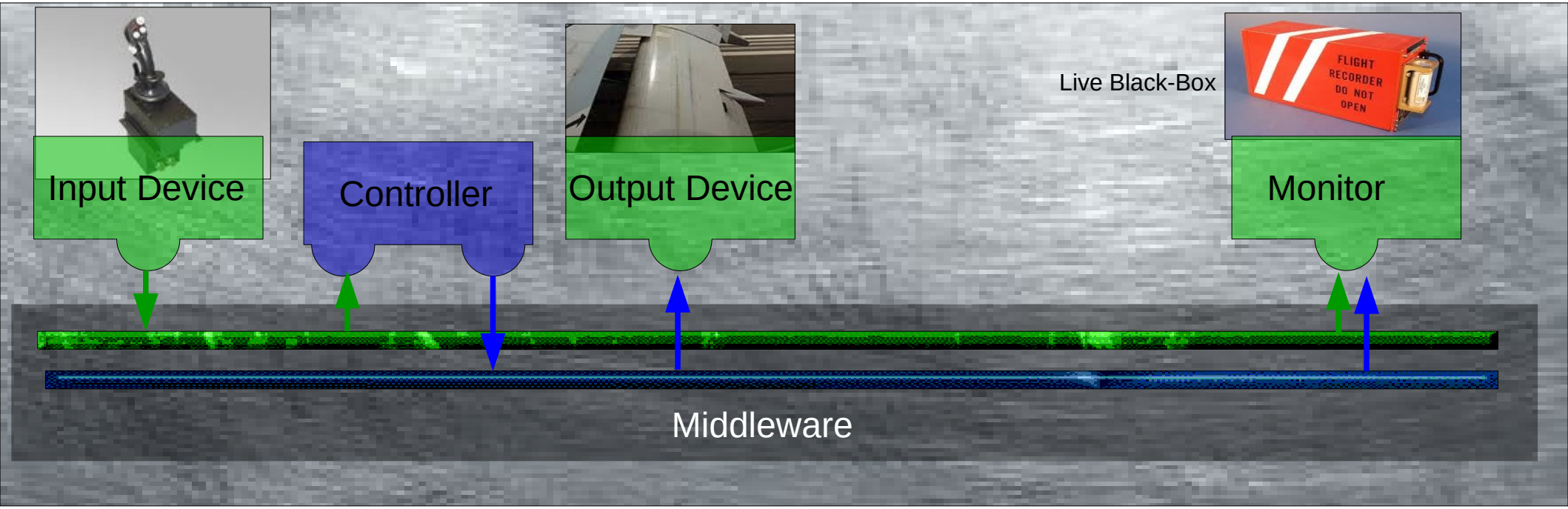
Communication Channels


Middleware
Topics ~=
Comm.
Channels
Publish
Publish
Publish
Subscriber-A
Subscriber-B
Subscriber-C
Event-A
Event-B
Only publishFromInterrupt
Resume
Thread-A
Resume
Thread-B
Thread-C
End-Of-Time
(150 Years)

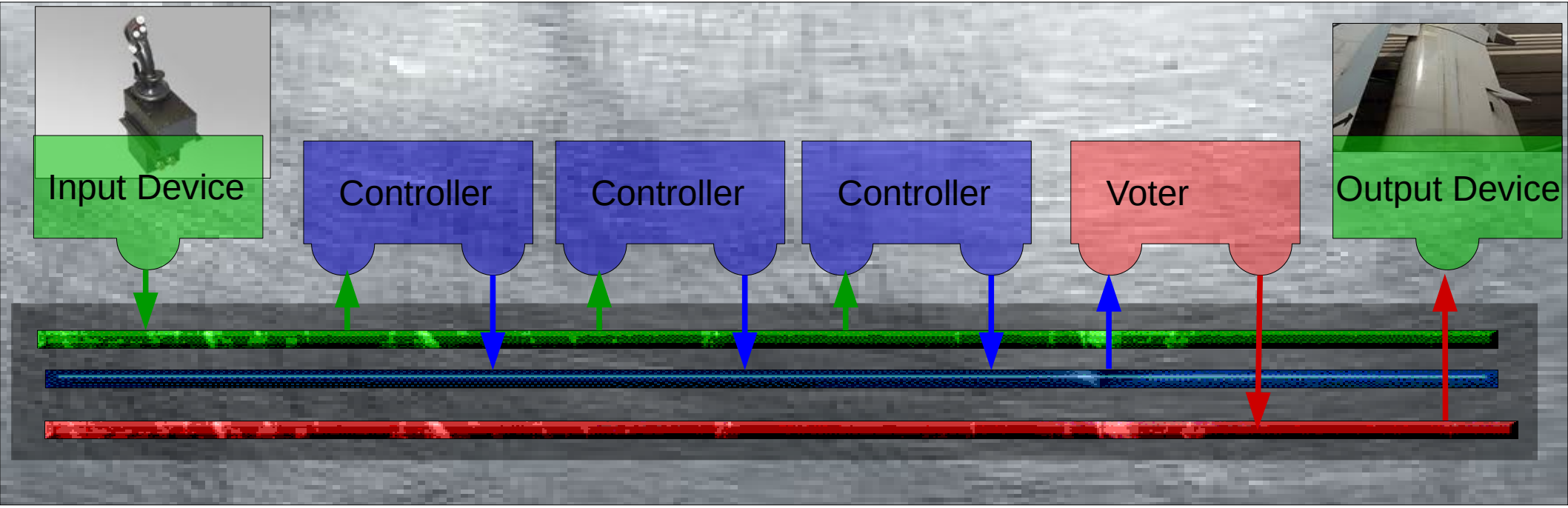
Dynamic Composition



Dynamic Composition



Dynamic Composition



Topic

```
Topic<Type>::Topic(int64_t id, const char* name, bool _onlyLocal = false)
```

- ID is given in the constructor
- Type is defined as a template parameter
- Name as string (for debugging)
- Flag onlyLocal for not forwarding it via gateway (see later)

Publisher

- No special registering necessary
- Just invoke Topic::publish method

```
publish(Type &msg, bool shallSendToNetwork = true)
```

  - msg: value to send
  - shallSendToNetwork: if the message should be forwarded via gateway (see later)

```
void publishFromInterrupt(void *any, size_t len = 0);
```

  - no thread safety

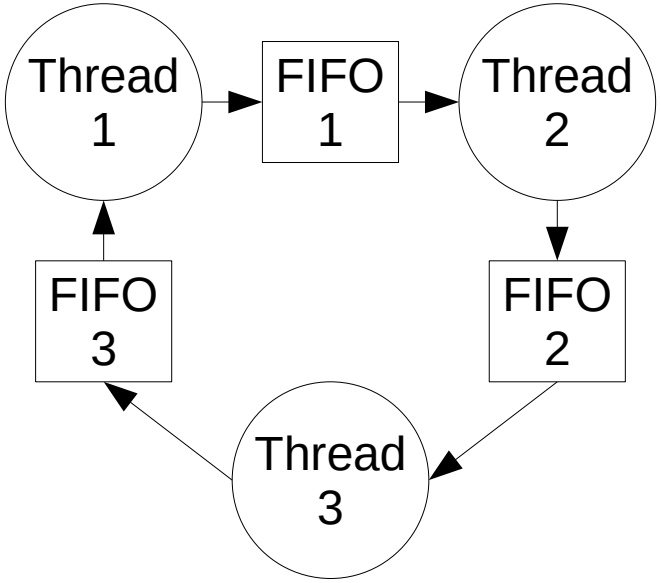
a) Subscriber Subclass

```
RODOS::Topic<int> myTopic(123, "MyTopic");

class MySubscriber : RODOS::SubscriberReceiver<int> {
public:
  MySubscriber() : RODOS::SubscriberReceiver(myTopic, "MySubscriber") {}

  uint32_t put(int& data) override {
    int* intData = reinterpret_cast<int*>(data);
    // process intData
  }
};

MySubscriber mySubscriber;
```

b) Passing Putter

```
RODOS::Topic<int> myTopic(123, "MyTopic");

RODOS::SyncFifo<int, 16> myFifo;

Rodos::Subscriber myFifoSubscriber(myTopic, myFifo, "MyFifoSubscriber");

RODOS::CommBuffer<int> myBuffer;

Rodos::Subscriber myBufferSubscriber(myTopic, myBuffer, "MyFifoSubscriber");
```

Exercise



- Create three Threads and three FIFOs for int values
- The Threads shall put data into and read data from the FIFO as shown in the diagram
- Every time a thread receives data from a FIFO, the value shall be printed (together with the thread‘s name), incremented, and forwarded to the other FIFO
- One thread starts the process by putting once a "1" to a FIFO
  - ➔ The effect: data is circulating in the circle, being incremented in each step.

Questions

- When should we use FIFOs and when CommBuffer?
- When should we use a semaphore?
- What could be dangerous when using semaphores?

Thank you for your attention!

Q&A session on

- Monday 14:00 UTC

Frank Flederer

E-Mail: frank.flederer@uni-wuerzburg.de

Tel.: +49 931 31-85120